

KAMKA IT Assessment Notes

Cloud & DevOps Intern (CI/CD & Infrastructure) - Summer 2026

Repository: kamka-cloud-devops-assessment

Architecture Summary

This submission implements a minimal three-tier Todo application focused on delivery lifecycle and operational clarity rather than application complexity.

- frontend: static HTML, CSS, and JavaScript served by Nginx
- api: Spring Boot REST API exposing Todo endpoints and health checks
- db: PostgreSQL persistence layer
- monitoring: Uptime Kuma watching the running stack
- ci/cd: GitHub Actions building, testing, publishing, and triggering deployment

Change flow from push to production:

1. A push triggers the GitHub Actions workflow.
2. The API test job runs first and fails loudly if the backend is broken.
3. If tests pass, Docker images for the API and frontend are built.
4. On push events, the images are pushed to Docker Hub with branch, SHA, and latest tags on the default branch.
5. On main, the deploy job runs and still documents the reproducible local-first production path through `docker-compose.prod.yml` and `scripts/deploy.sh`.
6. The production profile pulls published images instead of rebuilding them, which demonstrates reproducible deployment using the same service topology as development.

Key Decisions and Trade-offs

Static frontend: A static frontend keeps the focus on infrastructure while still showing reverse proxying, container separation, and frontend-to-API traffic.

Spring Boot API: Spring Boot provides a realistic backend shape with health checks, PostgreSQL integration, and a familiar build pipeline without unnecessary application complexity.

Docker Hub registry: Docker Hub public images reduce friction for reviewers because they can inspect and pull images immediately.

GitHub Actions: GitHub Actions keeps the pipeline close to the repository and clearly expresses the stages: test, build, publish, and deploy trigger.

Uptime Kuma: Uptime Kuma is lightweight and easy to demonstrate locally. It directly answers whether the frontend, API, and database are available.

Secrets handling: Secrets stay in local `.env` files or GitHub Actions secrets, while `.env.example` documents the required variables without exposing real credentials.

Dev / Prod Parity

Development and production use the same core runtime shape: Nginx frontend, Spring Boot API, PostgreSQL, and Uptime Kuma. The main difference is image source.

- development builds images locally with docker compose up --build
- production pulls versioned images from Docker Hub with docker-compose.prod.yml
- health checks, service dependencies, and monitoring stay consistent across both environments

Validation Performed

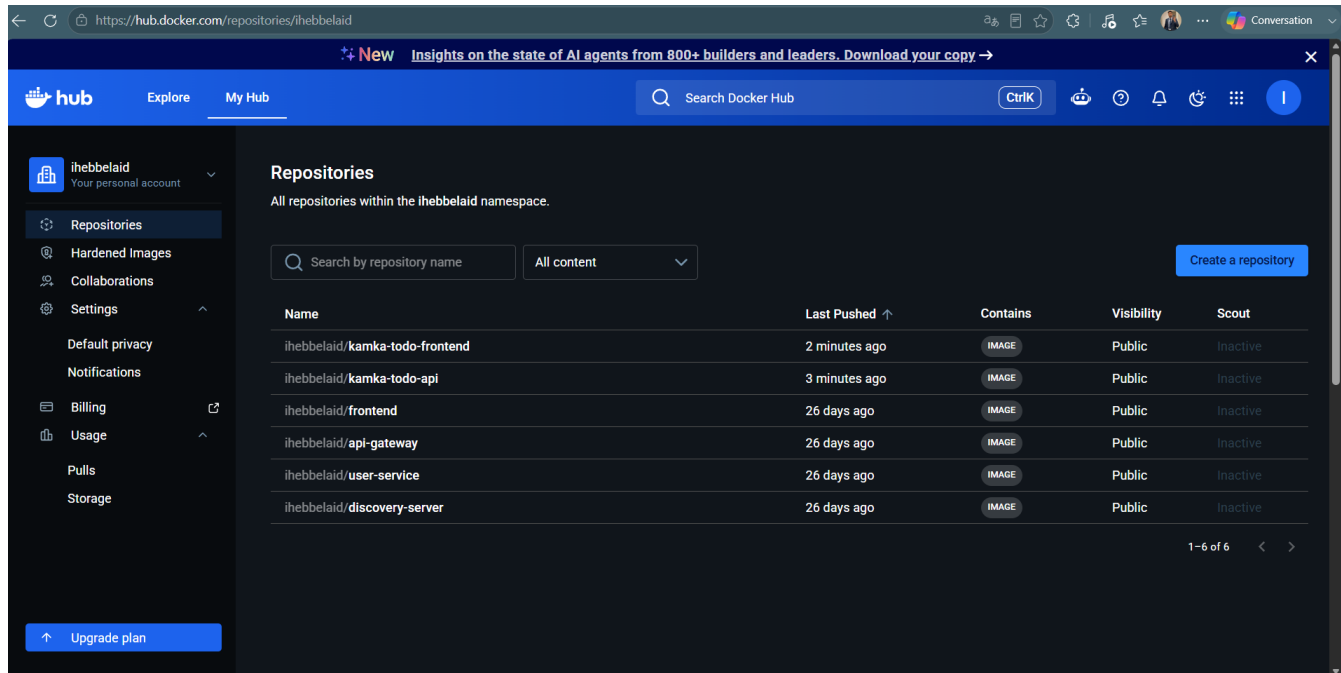
- docker compose up --build -d starts the development stack successfully
- docker compose -f docker-compose.prod.yml --env-file .env up -d starts the production profile successfully
- the frontend is reachable on <http://localhost:3000>
- Spring Boot health responds on <http://localhost:8081/actuator/health>
- PostgreSQL accepts connections
- Uptime Kuma is reachable on <http://localhost:3001> and monitors frontend, API, and PostgreSQL
- GitHub Actions completed successfully for test, build, image push, and deploy-stage signaling
- Docker Hub contains public repositories for kamka-todo-api and kamka-todo-frontend

Validation Screenshots

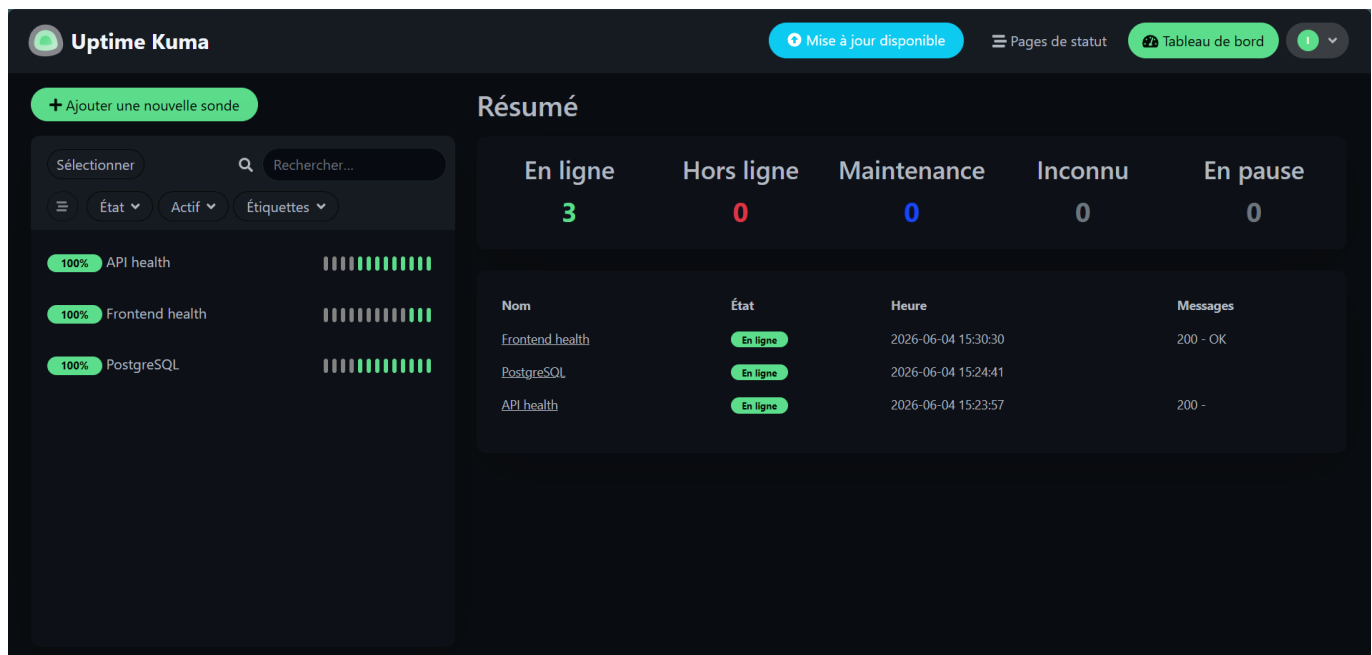
The most relevant validation screenshots are embedded below because the submission form does not provide separate screenshot upload slots.

The screenshot shows a GitHub Actions workflow run titled "Switch CI/CD from GHCR to Docker Hub #2". The workflow is triggered by a push to the main branch. The status is "Success" with a total duration of 3m 5s and 2 artifacts. The workflow steps are: test-api (20s), build-images (2m 27s), and deploy (9s). The build-images step is highlighted, showing a summary of the Docker build process. The summary includes a link to download the build record archive and import it into Docker Desktop's Builds view. The summary also includes a link to learn more about build records.

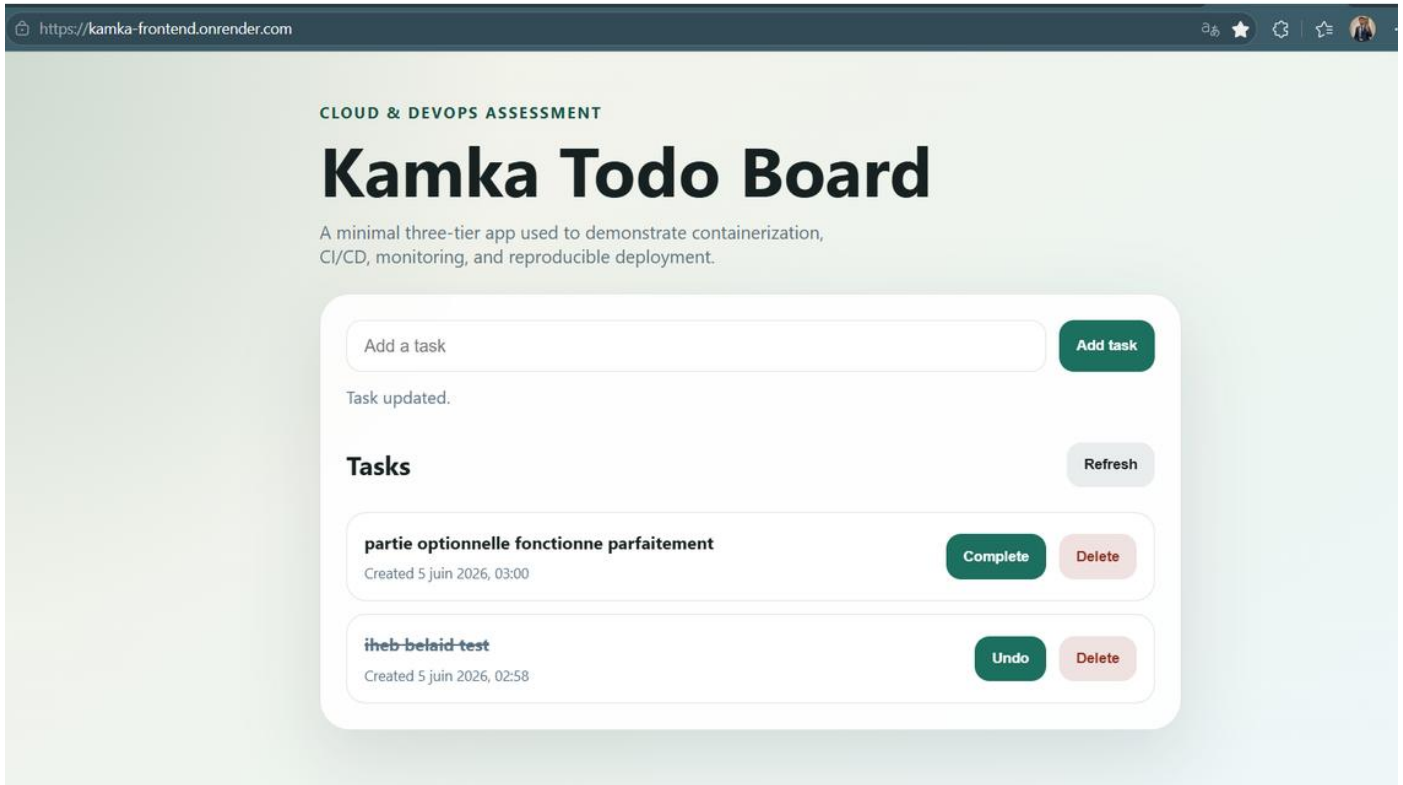
GitHub Actions workflow completed successfully



Docker Hub repositories published successfully



Uptime Kuma monitoring the full stack with three healthy checks



Optional bonus live deployment on Render: production overview and public frontend working end to end.

Known Limitations

- Uptime Kuma monitors are created manually on first boot rather than auto-provisioned.
- The optional Render deployment now exposes a public live host, while Docker Compose remains the primary reproducible environment.
- Render-managed HTTPS is active on the public frontend and API endpoints; the local stack intentionally stays HTTP-only.
- Rollback is manual through previously published image tags rather than automated.
- The application stays intentionally minimal and does not include authentication or advanced data migration handling.

Optional Bonus: Live Deployment on Render

- A public Render project now hosts kamka-api, kamka-frontend, and kamka-postgres in Frankfurt.
- The live frontend successfully creates, completes, reopens, refreshes, and deletes todos against the hosted API.
- The Render free tier remains subject to cold starts after inactivity and a 30-day PostgreSQL expiry window.
- Future improvements would focus on custom domains, backup automation, and rollback orchestration rather than basic hosting.